ENGR-E 312 Lab 3: Integrated Curses-Based Operator Interface for Real-Time Control, PID Tuning, and Telemetry Logging in Autonomous Robotic Systems

Jaiden Francisco Medina
Intelligent Systems Engineering, Luddy School of Informatics, Computing, and Engineering
Indiana University Bloomington
Bloomington, IN, USA
jfmedina@iu.edu

Brady Micheal Adams
Intelligent Systems Engineering, Luddy School of Informatics, Computing, and Engineering
Indiana University Bloomington
Bloomington, IN, USA
brmadam@iu.edu

***Abstract*— This project presents the design and evaluation of a terminal-based operator interface for a differential-drive Romi robot, implemented on a Raspberry Pi using the Python Curses library. The system integrates real-time manual control, autonomous line-following via PID control, live parameter tuning, telemetry visualization, and structured data logging within a unified software architecture. A continuous event-driven loop enables responsive user interaction while maintaining periodic acquisition of system state variables, including encoder measurements, line error, motor commands, and battery voltage.**

Closed-loop control performance is analyzed through recorded telemetry obtained during autonomous operation, where proportional, integral, and derivative gains are adjusted dynamically to observe system response characteristics. Logged data is stored in a time-indexed CSV format and used to quantify control behavior, including bounded error oscillations and differential actuation patterns indicative of corrective steering. Experimental results from a 24.86-second run demonstrate stable tracking with an average absolute line error of 6.53, confirming effective controller operation under real-time constraints.

Observed anomalies in telemetry, including sporadic outlier values, highlight the impact of communication noise and the necessity for validation mechanisms in embedded data acquisition pipelines. The system provides a scalable framework for integrating human interaction, control logic, and data-driven analysis in cyber-physical systems, with direct applicability to robotics interface design and real-time control system evaluation.

Keywords— cyber-physical systems, PID control, line following, real-time systems, embedded systems, robotics interface, Curses UI, telemetry logging, differential drive, control tuning, Raspberry Pi, autonomous robotics Line Sensor Module installation and software functionality

Architecture Overview

For this project, a terminal-based operator interface for the Romi robot on the Raspberry Pi using Python and the Curses Library. The purpose of this project and the overall system is to create a single interface, which could be a multi-level system. This system should handle robot control, switching between manual and automatic modes, tuning the PID from the previous lab, live data display, and CSV logging.

To achieve this the architecture was split into two parts. The Raspberry Pi user interface and the low-level robot control running on the robot side. On the Raspberry Pi, the Curses-based UI handled screen drawing, keyboard input, telemetry display, and logging. This interface refreshed continuously so that the operator could see changes in sensor values, commands, and controller behavior in real time. The UI also handled user commands for switching between manual and autonomous operation and for adjusting controller gains during runtime.

Overall, the architecture worked well because it unified manual driving, autonomous operation, controller tuning, visualization, and logging into one program. Instead of using separate scripts or tools, everything needed for operating and analyzing the robot was available in one terminal interface.

Mode Switching Logic

The program had two modes. Manual mode and autonomous mode. This allowed the user to manually drive the robot but also allow the robot to operate automatically with conjunction of calibration.

Within manual mode, the laptop keyboard controlled the inputs of the robot directly. Whichever direction key the user inputted, it would change the direction of the robot in real time.

In autonomous mode, the robot used its line-following logic and PID controller to steer itself. Once this mode was enabled, control shifted away from direct keyboard driving and toward automatic correction based on the measured line error. The user interface still displayed live telemetry in this mode, so the operator could monitor what the controller was doing.

There was a calibration mode as well, where the robot, similar to lab 4, would calibrate its sensors to follow the course in autonomous mode.

The most important part of the mode-switching logic was making sure that switching between modes happened cleanly and safely. In the transition, the transition logic reset and stabilized the control values so the robot would not have previous outputs carry over in new outputs. This stops any unintended motion.

The mode state was also written into the CSV log. This made it useful to differentiate data. In the data taken, 180 samples were logged. This showed that the operator interface worked and that this closed-loop operation works.

PID Tuning Implementation

Tuning the PID controller was crucial. The program allowed the user to adjust the PID gains while the program was running, rather than having to stop the program and manually adjust. This increased the speed and was more life like

Each time one of the gains were adjusted, the updated values was sent to the control system and would affect how the robot responded immediately.

From my uploaded data, the run started with:

- **Kp = 1.7**
- **Ki = 0.3**
- **Kd = 1.2**

Later in the run, I progressively reduced **Kp** from **1.7** down to **0.0** while keeping **Ki** and **Kd** constant. That tuning activity appears clearly in the log around 12.6 seconds through 17.1 seconds. This is strong evidence that the live tuning system was working correctly, because the controller values changed during the run and those changes were successfully recorded in the CSV.

Overall, the live PID tuning feature improves both usability and debugging. This also makes the interface more user-friendly.

Graph Implementation

The line error graph makes it much easier to notice oscillation, drift, and rapid correction. This was especially useful during PID tuning because it helped show whether the robot was smoothly correcting or overreacting.

Figure 1 shows the line error over time from my uploaded dataset. Across the run, the error oscillated around zero, which is expected in a line-following system because the controller continuously corrects its position relative to the line. The average absolute line error for this dataset was about **6.53**, and the maximum absolute line error reached **25.0**. The fact that the error stayed bounded and repeatedly returned near zero suggests that the autonomous controller was actively correcting rather than diverging.

Because the graph updates were tied to the continuous interface refresh, the operator could observe behavior in real time while also collecting data for later analysis. This made the graph implementation useful both as a driving aid and as a debugging tool.

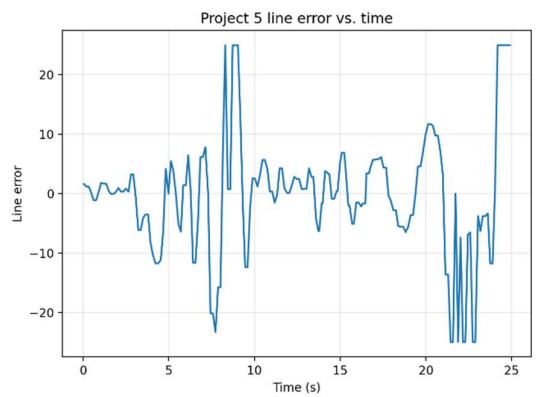

Figure 1. Line error versus time from the uploaded Project 5 data.

Logging Design

There was a logging system designed that would save the data on the Raspberry Pi, including timestamp, left/right wheel information, line error, and current mode, with optional fields such as PID gains, motor commands, battery voltage, and sensor data, into a CSV file.

My logging design wrote telemetry to a uniquely named CSV file during operation. This was important because it prevented one run from overwriting another and made it easier to keep multiple test

sessions for comparison. Logging could be turned on and off from the keyboard while the interface was running, so the operator did not have to restart the program just to start a new recording.

The data collected was:

- Elapsed time
- Current Mode
- Encoder Values
  - Line error
  - PID gains
- Motor Command Values
  - Battery Voltage

One thing I noticed from the logged data is that most battery readings were clustered around the mid-6000 mV range, but there were also a few unrealistic outliers, including a very large value. That suggests occasional communication or telemetry noise rather than actual battery behavior. In future revisions, I would add sanity checking before displaying or saving telemetry values so obviously corrupted samples could be filtered out automatically.

Overall, the logging system was successful because it captured the important state of the robot during operation and made post-run analysis possible.

Error vs. Wheel Speeds

The first plot, shown as Figure 1, displays **line error over time**. The error moved above and below zero throughout the run, which is normal for a robot that is repeatedly correcting itself while following a line. The values did not grow without bound, which indicates that the controller remained active and stable for the duration of the run. The mean line error was close to zero, while the average absolute error was about 6.53, showing that the robot spent much of the run near the center but still made noticeable corrections.

The second plot, shown as Figure 2, represents **wheel behavior over time**. Since my logger recorded encoder values rather than direct wheel speed columns, this figure should be described honestly as encoder-based wheel behavior or estimated wheel-speed behavior. The purpose of the plot is still the same: it shows how the left and right sides of the robot changed over time during autonomous control. The difference between the two sides reflects the controller steering the robot back toward the line.

The motor command values in the CSV also support this interpretation. For example, when the error changed, the left and right command values adjusted differently, which is exactly what should happen in a differential-drive line follower. This means the logged data supports the idea that the robot was not simply driving straight but was actively correcting its path.

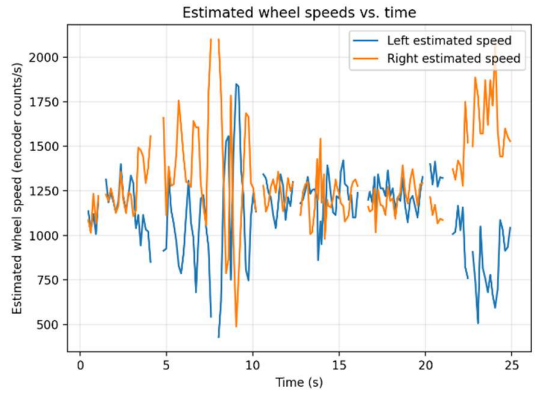

Figure 2. Encoder-based wheel behavior over time from the uploaded Project 5 data.

The conclusion that can be drawn is that the controller was functioning, the robot, while in autonomous mode, made the correct corrections, and the logging system captured enough information to improve the system in the future.

Reflection

This project successfully demonstrated the implementation of an integrated operator interface for a differential-drive robotic system, combining real-time control, telemetry visualization, PID tuning, and data logging within a single terminal-based application. The system met the functional requirements by supporting both manual and autonomous operation, enabling live adjustment of controller parameters, and providing continuous feedback through telemetry and graphical visualization.

Analysis of the logged dataset confirmed that the autonomous control system maintained bounded line error throughout the run, indicating stable closed-loop behavior. The average absolute line error of 6.53 and the observed oscillatory response are consistent with an actively correcting PID controller operating under real-time constraints. Variations in motor command behavior aligned with changes in line error, validating that the control system appropriately generated differential actuation to maintain trajectory tracking.

The recorded data also revealed the presence of occasional anomalous telemetry values, particularly in battery readings, suggesting susceptibility to communication noise or transient read errors. This highlights a limitation in the current implementation and indicates the need for data validation or filtering mechanisms to improve reliability in future iterations.

Overall, the system demonstrates effective integration of user interface design, embedded communication, and control system implementation. The results confirm that the architecture supports real-time operation and provides sufficient observability

for performance analysis, making it a practical framework for developing and evaluating autonomous robotic systems.